

Lista de exercícios 2

Projeto e análise de algoritmos

Heaps

1. Quais são os números mínimo e máximo de elementos em um heap de altura h ?
2. Mostre que um heap de n elementos tem altura $\lceil \lg n \rceil$.
3. Mostre que, em qualquer subárvore de um heap de máximo, a raiz da subárvore contém o maior valor que ocorre em qualquer lugar nessa subárvore.
4. Em que lugar de heap de máximo o menor elemento poderia residir, considerando que todos os elementos sejam distintos?
5. Um vetor que está em sequência ordenada é um heap de mínimo?
6. A sequência $\langle 33, 19, 20, 15, 13, 10, 2, 13, 16, 12 \rangle$ é um heap de máximo?
7. Ilustre a operação de MAXIMIZA-HEAP ($A, 3$) sobre o vetor $A = \langle 27, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0 \rangle$.
8. Começando com o procedimento MAXIMIZA-HEAP, escreva o pseudocódigo para o procedimento MINIMIZA-HEAP (A, i), que executa a manipulação correspondente sobre um heap de mínimo.
9. Qual é o efeito de chamar MAXIMIZA-HEAP(A, i) quando o elemento $A[i]$ é maior que seus filhos?
10. Ilustre a operação de HEAPSORT sobre o vetor $A = \langle 5, 13, 2, 25, 7, 17, 20, 8, 4 \rangle$.
11. Suponha que os objetos em uma fila de prioridade máxima sejam apenas chaves. Ilustre a operação de EXTRAI-MAX-MAX-HEAP sobre o heap $A = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$.
12. Suponha que os objetos em uma fila de prioridade máxima sejam apenas chaves. Ilustre a operação de INSERE-MAX-HEAP($A, 10, 15$) sobre o heap $A = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$.
13. Mostre como implementar uma fila “primeiro a entrar, primeiro a sair” com uma fila de prioridade. Mostre como implementar uma pilha com uma fila de prioridades.

Ordenação em tempo linear

1. Ilustre a operação de ORDENA-POR-CONTAGEM sobre o vetor $A = \langle 6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2 \rangle$.
2. Prove o seguinte invariante de laço para ORDENA-POR-CONTAGEM:
No início de cada iteração do laço for das linhas 11–13, o último elemento em A com valor i que ainda não foi copiado para B pertence a $B[C[i]]$.
3. Suponha que o vetor sendo ordenado contenha apenas inteiros no intervalo de 0 a k e que não haja dados satélites a serem movidos com essas chaves. Modifique a ordenação por contagem para que use apenas os vetores A e C , colocando o resultado ordenado de volta para o vetor A , em vez de um novo vetor B .
4. Ilustre a operação de ORDENA-POR-DÍGITO sobre a seguinte lista de palavras em inglês: COW, DOG, SEA, RUG, ROW, MOB, BOX, TAB, BAR, EAR, TAR, DIG, BIG, TEA, NOW, FOX.

5. Quais dos seguintes algoritmos de ordenação são estáveis: ordenação por inserção, ordenação por intercalação, ordenação por heap e quicksort? Forneça um esquema simples que torne estável qualquer algoritmo de ordenação. Quanto tempo e espaço adicionais seu esquema requer?
6. Ilustre a operação de ORDENA-POR-BALDE no vetor $A = \langle 0,79, 0,13, 0,16, 0,64, 0,39, 0,20, 0,89, 0,53, 0,71, 0,42 \rangle$.
7. Explique por que o tempo de execução do pior caso para a ordenação por balde é $\Theta(n^2)$. Qual é a alteração simples no algoritmo que preserva seu tempo de execução linear do caso médio e torna seu tempo de execução do pior caso igual a $O(n \lg n)$?

Algoritmos aleatorizados

1. Descreva uma implementação do procedimento ALEATÓRIO(a, b) que só faça chamadas a ALEATÓRIO($0, 1$). Qual é o tempo de execução esperado de seu procedimento, em função de a e b ?
2. Em CONTRATAR-AUXILIAR, supondo que os candidatos sejam apresentados em ordem aleatória, qual é a probabilidade de você contratar exatamente uma vez? Qual é a probabilidade de você contratar exatamente n vezes?
3. Quantas pessoas devem estar em uma sala antes que a probabilidade de alguém ter a mesma data de aniversário que você seja no mínimo $1/2$? Quantas pessoas devem estar presentes antes de a probabilidade de no mínimo duas pessoas fazerem aniversário no dia 7 de setembro ser maior que $1/2$?
4. Quantas pessoas devem estar em uma sala antes que a probabilidade de duas pessoas terem a mesma data de aniversário seja no mínimo $0,99$? Para essa quantidade de pessoas, qual é o número esperado de pares de pessoas que fazem aniversário no mesmo dia do ano?

Respostas de alguns exercícios (em inglês)

Heaps

1. Quais são os números mínimo e máximo de elementos em um heap de altura h ?
Since a heap is an almost-complete binary tree (complete at all levels except possibly the lowest), it has at most $2^{h+1} - 1$ elements (if it is complete) and at least $2^h - 1 + 1 = 2^h$ elements (if the lowest level has just 1 element and the other levels are complete).
2. Mostre que um heap de n elementos tem altura $\lceil \lg n \rceil$.
Given an n -element heap of height h , we know from Exercise 6.1-1 that
$$2^h \leq n \leq 2^{h+1} - 1 < 2^{h+1}.$$

Thus, $h \leq \lg n < h + 1$. Since h is an integer, $h = \lfloor \lg n \rfloor$ (by definition of $\lfloor \cdot \rfloor$).

3. Mostre que, em qualquer subárvore de um heap de máximo, a raiz da subárvore contém o maior valor que ocorre em qualquer lugar nessa subárvore.

Assume that the claim is false—i.e., that there is a subtree whose root is not the largest element in the subtree. Then the maximum element is somewhere else in the subtree, possibly even at more than one location. Let m be the index at which the maximum appears (the lowest such index if the maximum appears more than once). Since the maximum is not at the root of the subtree, node m has a parent. Since the parent of a node has a lower index than the node, and m was chosen to be the smallest index of the maximum value, $A[\text{PARENT}(m)] < A[m]$. But by the max-heap property, we must have $A[\text{PARENT}(m)] \geq A[m]$. So our assumption is false, and the claim is true.

4. Em que lugar de heap de máximo o menor elemento poderia residir, considerando que todos os elementos sejam distintos?

The smallest element must reside in a leaf.

5. Um vetor que está em sequência ordenada é um heap de mínimo?

Yes, an array in sorted order is a min-heap, since $A[i] \leq A[2i]$ where $2i \leq n$, and $A[i] \leq A[2i + 1]$ where $2i + 1 \leq n$.

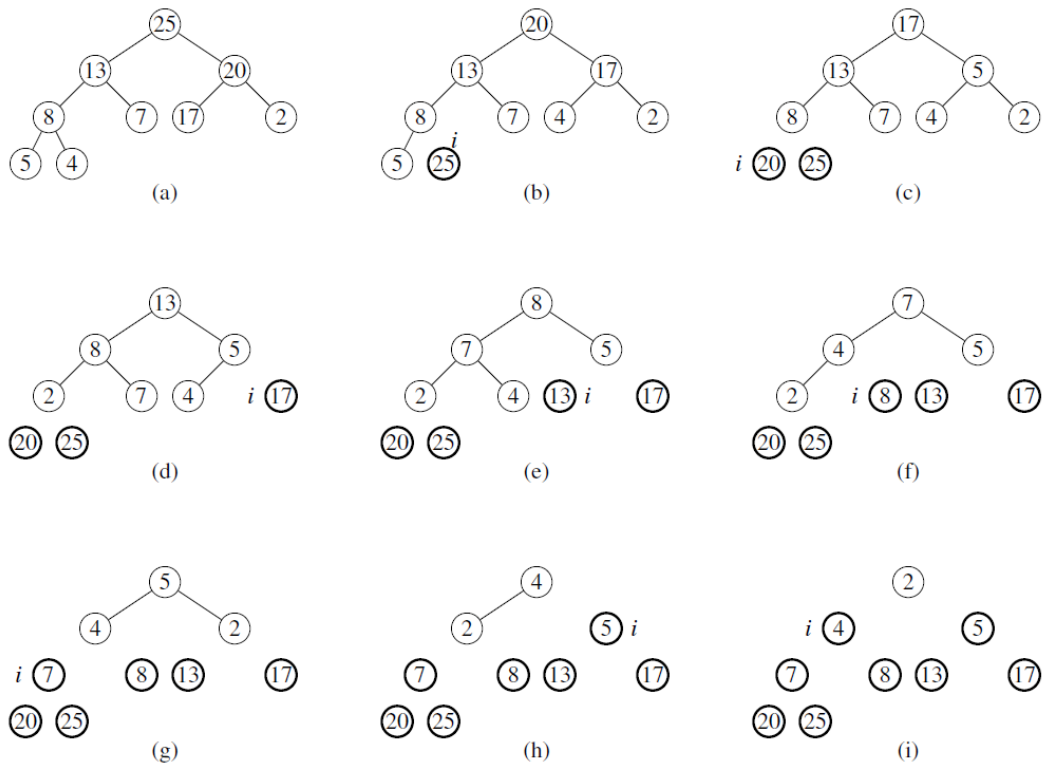
6. A sequência $\langle 33, 19, 20, 15, 13, 10, 2, 13, 16, 12 \rangle$ é um heap de máximo?

No, this array is not a max-heap. Element 15 has 13 and 16 as children, and $16 > 15$.

9. Qual é o efeito de chamar $\text{MAXIMIZA-HEAP}(A, i)$ quando o elemento $A[i]$ é maior que seus filhos?

The heap doesn't change because line 8 of MAX-HEAPIFY finds that $\text{largest} = i$.

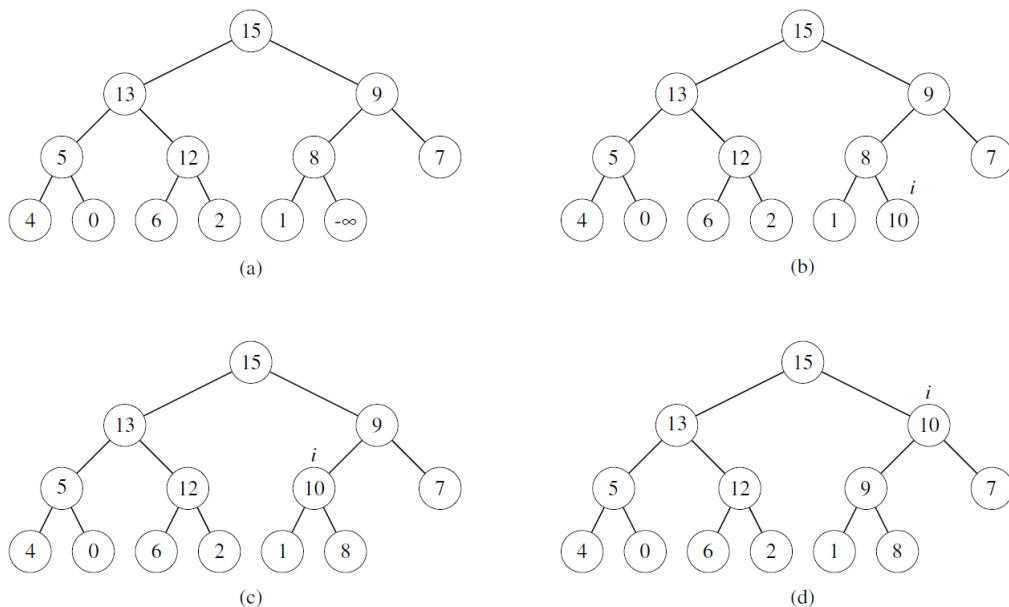
10. Ilustre a operação de HEAPSORT sobre o vetor $A = \langle 5, 13, 2, 25, 7, 17, 20, 8, 4 \rangle$.



A

2	4	5	7	8	13	17	20	25
---	---	---	---	---	----	----	----	----

12. Suponha que os objetos em uma fila de prioridade máxima sejam apenas chaves. Ilustre a operação de $\text{INSERE-MAX-HEAP}(A, 10, 15)$ sobre o heap $A = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$.



13. Mostre como implementar uma fila “primeiro a entrar, primeiro a sair” com uma fila de prioridade. Mostre como implementar uma pilha com uma fila de prioridades.

For a stack, use a max-priority queue and keep a counter c of how many objects have been pushed onto the stack, initially 0. To push, call INSERT with the current value of c as the key, and then increment c . To pop, just call MAXIMUM.

For a queue, use a min-priority queue, again keeping the counter c , initially 0. To enqueue, do the same as pushing: call INSERT with the current value of c as the key, and then increment c . To dequeue, call MINIMUM.

Ordenação em tempo linear

2. Prove o seguinte invariante de laço para ORDENA-POR-CONTAGEM:

No início de cada iteração do laço for das linhas 11–13, o último elemento em A com valor i que ainda não foi copiado para B pertence a $B[C[i]]$.

Loop invariant: At the start of each iteration of the **for** loop of lines 11–13, the last element in A with value i that has not yet been copied into B belongs in $B[C[i]]$.

Initialization: Initially, no elements in A have been copied into B , so that the last element in A with value i that has not been copied into B is just the last element in A with value i . Since there are $C[i]$ elements in A with value less than or equal to i , this last element in A with value i belongs in $B[C[i]]$.

Maintenance: Let $A[j] = i$ in a given iteration of the **for** loop of lines 11–13. By the loop invariant, $A[j]$ belongs in $B[C[i]]$. Let $m = \max\{l : l < j \text{ and } A[l] = i\}$ be the index of the rightmost element of A with value i that occurs before $A[j]$. Then $A[m]$ should go into the position of B immediately before where $A[j]$ goes, that is, $A[m]$ should go into position $C[i] - 1$. Decrementing $C[A[j]]$ in line 13 causes that to happen in the later iteration when $j = m$.

Termination: The loop terminates after n iterations. At that time, each element of A has been copied into its correct location in B .

3. Suponha que o vetor sendo ordenado contenha apenas inteiros no intervalo de 0 a k e que não haja dados satélites a serem movidos com essas chaves. Modifique a ordenação por contagem para que use apenas os vetores A e C , colocando o resultado ordenado de volta para o vetor A , em vez de um novo vetor B .

Count how many of each key there are and then just refill A with the correct number of each key.

COUNTING-SORT-KEYS-ONLY(A, n, k)

let $C[0:k]$ be a new array

for $i = 0$ **to** k

$C[i] = 0$

for $j = 1$ **to** n

$C[A[j]] = C[A[j]] + 1$

// $C[i]$ now contains the number of elements equal to i .

$j = 1$

for $i = 0$ **to** k

for $l = 1$ **to** $C[i]$

$A[j] = i$

$j = j + 1$

return A

5. Quais dos seguintes algoritmos de ordenação são estáveis: ordenação por inserção, ordenação por intercalação, ordenação por heap e quicksort? Forneça um esquema simples que torne estável qualquer algoritmo de ordenação. Quanto tempo e espaço adicionais seu esquema requer?

Insertion sort is stable. When inserting $A[j]$ into the sorted sequence $A[1:j-1]$, the procedure compares $A[j]$ with each $A[i]$, starting with $i = j-1$ and going down to $i = 1$. It continues as long as $A[j] < A[i]$.

Merge sort as defined is stable, because when two elements compared are equal, the tie is broken by taking the element from array L , which keeps them in the original order.

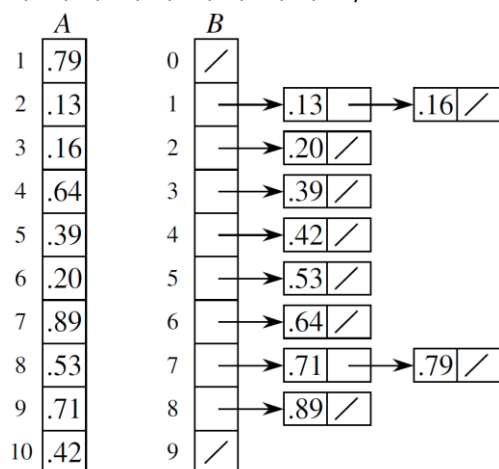
Heapsort and quicksort are not stable.

One scheme that makes a sorting algorithm stable is to store the index of each element (the element's place in the original ordering) with the element. When comparing two elements, compare them by their values and break ties by their indices.

Additional space requirements: For n elements, their indices are $1, \dots, n$. Each can be written in $\lg n + 1$ bits, so together they take $O(n \lg n)$ additional space.

Additional time requirements: The worst case is when all elements are equal. The asymptotic time does not change because each comparison takes an additional constant amount of time.

6. Ilustre a operação de ORDENA-POR-BALDE no vetor $A = \langle 0,79, 0,13, 0,16, 0,64, 0,39, 0,20, 0,89, 0,53, 0,71, 0,42 \rangle$.



7. Explique por que o tempo de execução do pior caso para a ordenação por balde é $\Theta(n^2)$. Qual é a alteração simples no algoritmo que preserva seu tempo de execução linear do caso médio e torna seu tempo de execução do pior caso igual a $O(n \lg n)$?

The worst-case running time for the bucket-sort algorithm occurs when the assumption of uniformly distributed input does not hold. If, for example, all the input ends up in the first bucket, then in the insertion sort phase it needs to sort all the input, which takes $\Theta(n^2)$ time in the worst case.

A simple change that will preserve the linear expected running time and make the worst-case running time $O(n \lg n)$ is to use a worst-case $O(n \lg n)$ -time algorithm, such as merge sort or heapsort, instead of insertion sort when sorting the buckets.

Algoritmos aleatorizados

2. Em CONTRATAR-AUXILIAR, supondo que os candidatos sejam apresentados em ordem aleatória, qual é a probabilidade de você contratar exatamente uma vez? Qual é a probabilidade de você contratar exatamente n vezes?

Since HIRE-ASSISTANT always hires candidate 1, it hires exactly once if and only if no candidates other than candidate 1 are hired. This event occurs when candidate 1 is the best candidate of the n , which occurs with probability $1/n$.

HIRE-ASSISTANT hires n times if each candidate is better than all those who were interviewed (and hired) before. This event occurs precisely when the list of ranks given to the algorithm is $\langle 1, 2, \dots, n \rangle$, which occurs with probability $1/n!$.

3. Quantas pessoas devem estar em uma sala antes que a probabilidade de alguém ter a mesma data de aniversário que você seja no mínimo $1/2$? Quantas pessoas devem estar presentes antes de a probabilidade de no mínimo duas pessoas fazerem aniversário no dia 7 de setembro ser maior que $1/2$?

We make three observations:

1. Candidate 1 is always hired.
2. The best candidate, i.e., the one whose rank is n , is always hired.
3. If the best candidate is candidate 1, then that is the only candidate hired.

Therefore, in order for HIRE-ASSISTANT to hire exactly twice, candidate 1 must have rank $i \leq n-1$ and all candidates whose ranks are $i+1, i+2, \dots, n-1$ must be interviewed after the candidate whose rank is n . (When $i = n-1$, this second condition vacuously holds.)

Let E_i be the event in which candidate 1 has rank i ; clearly, $\Pr\{E_i\} = 1/n$ for any given value of i .

Letting j denote the position in the interview order of the best candidate, let F be the event in which candidates $2, 3, \dots, j-1$ have ranks strictly less than the rank of candidate 1. Given that event E_i has occurred, event F occurs when the best candidate is the first one interviewed out of the $n-i$ candidates whose ranks are $i+1, i+2, \dots, n$. Thus, $\Pr\{F \mid E_i\} = 1/(n-i)$.

Our final event is A , which occurs when HIRE-ASSISTANT hires exactly twice. Noting that the events $E_1, E_2, \dots, E_n$ are disjoint, we have

$$\begin{aligned} A &= F \cap (E_1 \cup E_2 \cup \dots \cup E_{n-1}) \\ &= (F \cap E_1) \cup (F \cap E_2) \cup \dots \cup (F \cap E_{n-1}) . \end{aligned}$$

and

$$\Pr\{A\} = \sum_{i=1}^{n-1} \Pr\{F \cap E_i\} .$$

By equation (C.16),

$$\begin{aligned} \Pr\{F \cap E_i\} &= \Pr\{F \mid E_i\} \Pr\{E_i\} \\ &= \frac{1}{n-i} \cdot \frac{1}{n} , \end{aligned}$$

and so

$$\begin{aligned}\Pr\{A\} &= \sum_{i=1}^{n-1} \frac{1}{n-i} \cdot \frac{1}{n} \\ &= \frac{1}{n} \sum_{i=1}^{n-1} \frac{1}{n-i} \\ &= \frac{1}{n} \left(\frac{1}{n-1} + \frac{1}{n-2} + \cdots + \frac{1}{1} \right) \\ &= \frac{1}{n} \cdot H_{n-1} ,\end{aligned}$$

where H_{n-1} is the n th harmonic number.